WEEK 2 PROGRESS REPORT

CS - 302

Bermudo, Jeanne Clarrise T.
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario Angeles City, Philippines 2009
+63 968 247 8019
jtbermudo1@student.hau.edu.ph

Magat, Maria Josephine M.
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario Angeles City, Philippines 2009
+63 992 853 2885
mmmagat1@student.hau.edu.ph

Pineda, Mary Alexa Ysabelle V.
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario Angeles City, Philippines 2009
+63 909 204 2084
mvpineda@student.hau.edu.ph

Rebusa, Amber Kaia J.
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario Angeles City, Philippines 2009
+63 918 666 3016
ajrebusa1@student.hau.edu.ph

1. DATA AQUISITION AND ANALYSIS

The dataset used in this project was obtained from the Hugging Face dataset repository. Specifically, the Bitext Retail E-commerce Customer Support Dataset was used as the primary source of training data for the intent classification component of the system. This dataset was selected due to its relevance to the delivery service domain, containing customer support intents that closely align with the scope of this study. The dataset was downloaded and prepared using the script data/get_data.py, which automates the retrieval process and stores the raw dataset in the data/raw directory for subsequent preprocessing and model training.

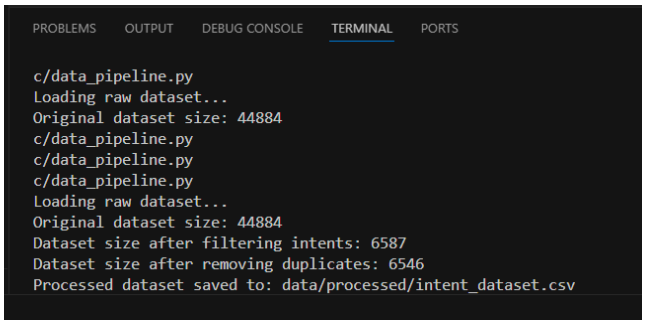
```
from datasets import load_dataset
dataset =
load_dataset("bitext/Bitext-retail-ecommerce-llm-chatbot-training-dataset")
dataset["train"].to_csv("data/raw/bitext_retail_dataset.csv")
```

Since the dataset contains various e-commerce intents beyond delivery support, a filtering process was applied to retain only the intents relevant to the scope of this study, such as delivery tracking, delivery issues, delivery time, and shipment-related concerns. Intents that fall outside of the delivery service domain were excluded to ensure that the model is trained on focused and relevant data. The dataset preprocessing and filtering steps were implemented in src/data_pipeline.py, which handles the necessary data cleaning and preparation prior to model training.

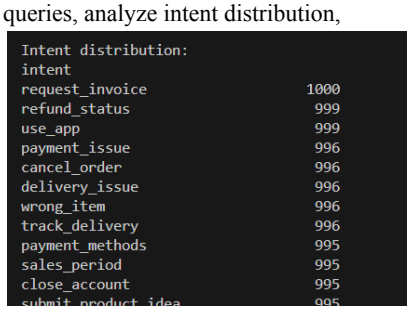
```
delivery_intents = [
    "track_order", "track_delivery", "delivery_issue",
    "delivery_time", "damaged_delivery", "missing_item",
    "shipping_costs" ]
df = df[df["intent"].isin(delivery_intents)]
df = df.drop_duplicates()

output_path = "data/processed/intent_dataset.csv"
```

The pipeline loads the dataset, filters the relevant intents, removes duplicate entries, and formats the text and label columns to ensure consistency across all samples. This script processes the raw dataset and produces a cleaned, structured dataset stored in data/processed/intent_dataset.csv, which serves as the final input for the model training stage.



Exploratory Data Analysis (EDA) was performed to inspect the dataset and understand the distribution of customer queries across the filtered intents. The analysis included examining example messages, analyzing intent distribution, and verifying the dataset size after filtering. These were performed using the script src/inspect_dataset.py, which was used to examine sample queries, analyze intent distribution,



2. INITIAL METRICS

The NLP component of the chatbot was implemented using BERT for intent classification. The model was fine tuned using the processed dataset. The training process involved several steps. First, user messages were tokenized using the BERT tokenizer to convert text into a format the model can process. Next, the intent labels were encoded so they could be used for supervised learning. This encoding step ensures that each unique intent is mapped to a numerical value that the model can interpret during training.

```
from transformers import BertTokenizer, BertForSequenceClassification
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=len(label_list))
```

The BERT classifier was then trained on the prepared dataset and evaluated using a validation set to measure its performance. After one epoch of training, the model achieved the following evaluation results:

Metric	Value
Accuracy	99.16%
Precision	99.20%
Recall	99.16%
F1 Score	99.16%

3. NLP PROTOTYPE

To validate the functionality of the NLP component, a prototype script located in src/predict.py was implemented.

```
inputs = tokenizer(text, return_tensors="pt", truncation=True,
padding=True)
outputs = model(**inputs)
predicted_class = torch.argmax(outputs.logits, dim=1).item()
```

This script loads the trained BERT intent classification model and allows the system to process user messages and predict the corresponding customer support intent. The prototype simulates real customer queries and demonstrates how the system interprets user messages related to delivery services. The model successfully classifies the intent of each query, which confirms that the trained model can correctly interpret delivery-related customer messages.

```
Customer message: there's a problem with our delivery
Predicted intent: delivery_issue
Customer message: our items came broken
Predicted intent: damaged_delivery
Customer message: where is our package?
Predicted intent: track_delivery
Customer message: how long will it take?
Predicted intent: track_delivery
Customer message: how much would shipping cost?
Predicted intent: shipping_costs
Customer message: where is my order?
Predicted intent: track_order
Customer message: the package you delivered has missing items.
Predicted intent: missing_item
Customer message: the package is not complete.
Predicted intent: missing_item
```

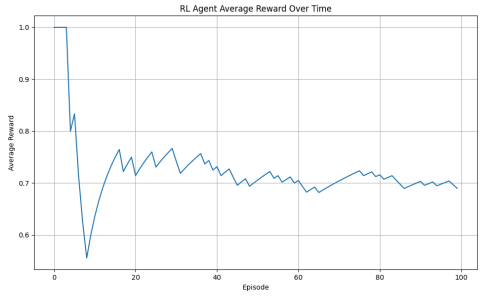
This demonstrates that the NLP model can accurately classify delivery-related intents from natural language queries. This functionality forms the core understanding component of the chatbot system.

4. RL PROTOTYPE

A reinforcement learning component was implemented using a multi armed bandit approach to optimize response selection within the chatbot. For each detected intent, the system contains multiple possible responses. The RL agent selects one of these responses and receives a reward signal based on simulated user feedback. In the prototype, the reward structure is defined so that a response that successfully resolves the user's issue receives a reward of 1, while a response that does not resolve the issue receives a reward of 0. Because real user feedback is not yet available, a simulated environment was used where each response is assigned a predefined probability of success. The RL agent interacts with this environment and updates its response selection strategy based on the rewards it observes, allowing it to gradually learn which responses are more effective. To observe the behavior of the RL agent, a learning curve was generated by plotting the average reward across interaction episodes.

```
true_reward_probabilities = [0.7, 0.5, 0.3]
action = agent.select_action()
prob = true_reward_probabilities[action]
reward = 1 if random.random() < prob else 0
agent.update(action, reward)

plt.plot(avg_rewards)
plt.title("RL Agent Average Reward Over Time")
plt.xlabel("Episode")
plt.ylabel("Average Reward")
```



The graph shows the average reward obtained by the agent over time as it explores different response options. Early episodes show fluctuations due to exploration, while later episodes stabilize as the agent identifies responses with higher success probabilities. Although the rewards are simulated, the learning curve demonstrates the functioning of the reinforcement learning framework and provides an initial view of the agent's learning behavior.

5. CURRENT PROGRESS

The following milestones have been achieved:

- Dataset acquisition and cleaning
- Exploratory data analysis
- BERT intent classification model prototype
- Initial model training and evaluation
- Reinforcement learning agent stub
- Early RL learning curve generation

These components demonstrate that the core NLP and RL structures of the system are functioning as intended.

6. CNN COMPONENT

To satisfy the CNN requirement of the project, a Text-CNN model will be implemented in the final stage as an additional deep learning experiment for intent classification. While the current prototype uses a BERT-based model as the primary NLP component, the Text-CNN will process customer queries using one-dimensional convolutional filters applied to text embeddings. The CNN model will be trained using the same processed dataset and evaluation setup used for the BERT classifier to ensure a fair comparison between architectures. By evaluating the Text-CNN alongside the BERT model using metrics such as accuracy, precision, recall, and F1 score, the project will be able to analyze the effectiveness of convolution-based text classification compared to transformer-based approaches while maintaining the reinforcement learning component for response optimization.